



Building Serverless Architectures and Microservices

AWS Academy Cloud
Architecting



Introduction

Building Serverless Architectures and Microservices

Module objectives

This module prepares you to do the following:

- Define serverless architectures.
- Identify the characteristics of microservices.
- Architect a serverless solution with AWS Lambda.
- Define how containers are used in Amazon Web Services (AWS).
- Describe the types of workflows that AWS Step Functions supports.
- Describe a common architecture for Amazon API Gateway.
- Use the AWS Well-Architected Framework principles when building serverless architectures.

Module overview

Presentation sections

- Thinking serverless
- Architecting serverless microservices
- Building serverless architectures with AWS Lambda
- Building microservice applications with AWS container services
- Orchestrating microservices with AWS Step Functions
- Extending serverless architectures with Amazon API Gateway
- Applying AWS Well-Architected Framework principles to microservices and serverless

Demos

- Using AWS Lambda with Amazon S3
- Running a Container

Activity

- Decomposing a Monolithic Application with AWS API Gateway

Knowledge checks

- 10-question knowledge check
- Sample exam question

Hands-on labs in this module

Guided labs

- Implementing a Serverless Architecture on AWS
- Breaking a Monolithic Node.js Application into Microservices (Optional)

Challenge (Café) lab

- Implementing a Serverless Architecture for the Café



As a cloud architect building microservices:



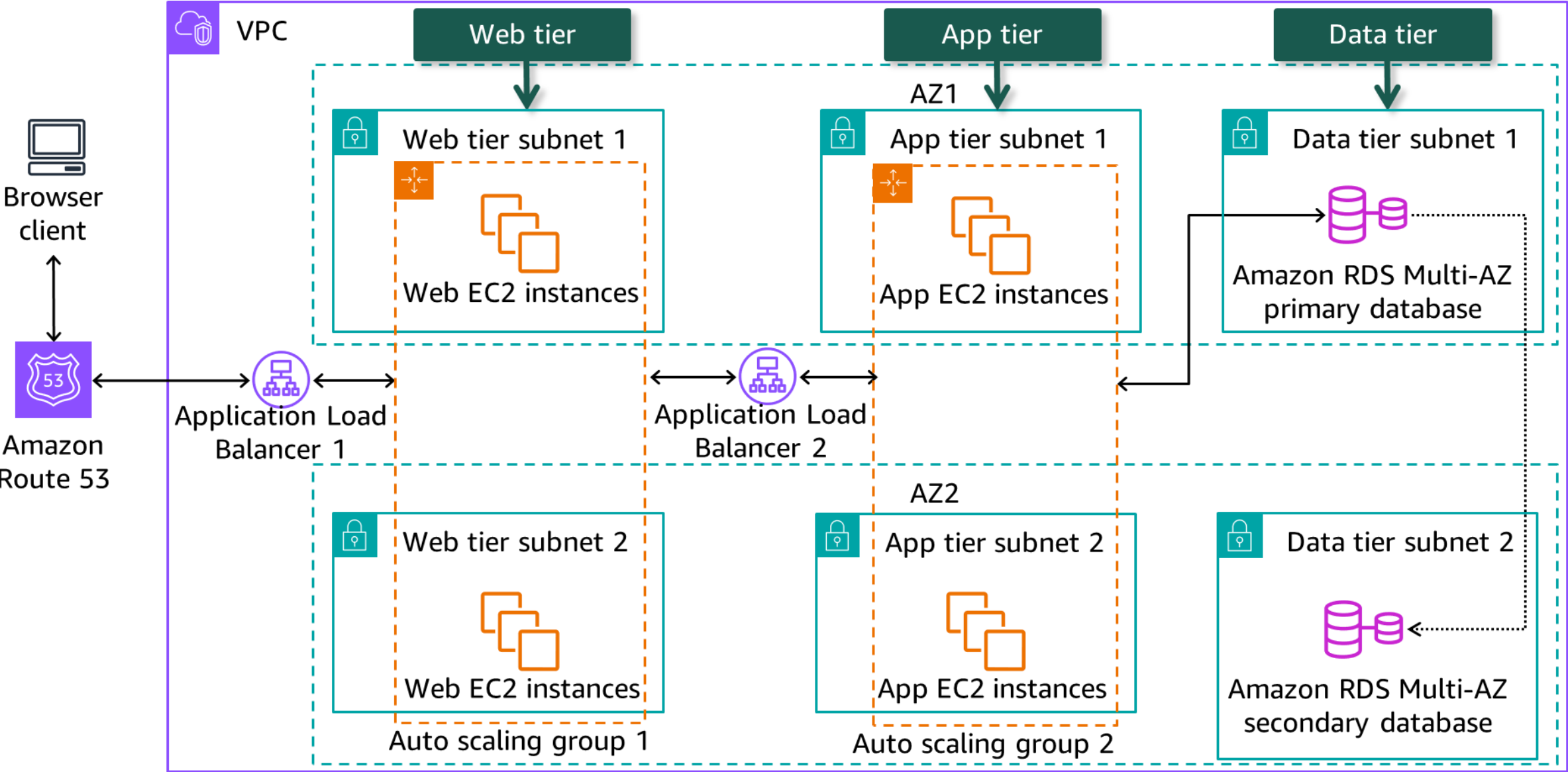
- I need to recognize when to choose AWS serverless architectures and which services to choose for each use case so that I can build a performance- and cost-optimized solution.
- I need to apply event-driven architectures with microservices so that I can build scalable and resilient microservice architectures.
- I need to know when to use workflow orchestrations so that my microservice architectures can handle workflow failures and require minimal manual intervention.



Thinking serverless

Building Serverless Architectures and Microservices

Web app three-tier design in a VPC



Benefits of AWS serverless



No server management



Continuous scaling



Pay-for-value services



Built-in high availability



Suitable for event-driven and
microservice architectures

AWS serverless services

Compute



AWS Lambda and
Lambda@Edge



AWS Fargate

Authentication



Amazon Cognito

Web hosting



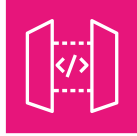
AWS Amplify

Content delivery



Amazon CloudFront

Application integration



Amazon API
Gateway



AWS AppSync



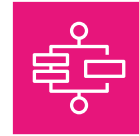
Amazon Simple
Notification Service
(Amazon SNS)



Amazon Simple
Queue Service
(Amazon SQS)



Amazon
EventBridge



AWS Step
Functions

Data stores



Amazon S3



Amazon Elastic
File System
(Amazon EFS)



Amazon
DynamoDB



Amazon Neptune
Serverless



Amazon Aurora
Serverless

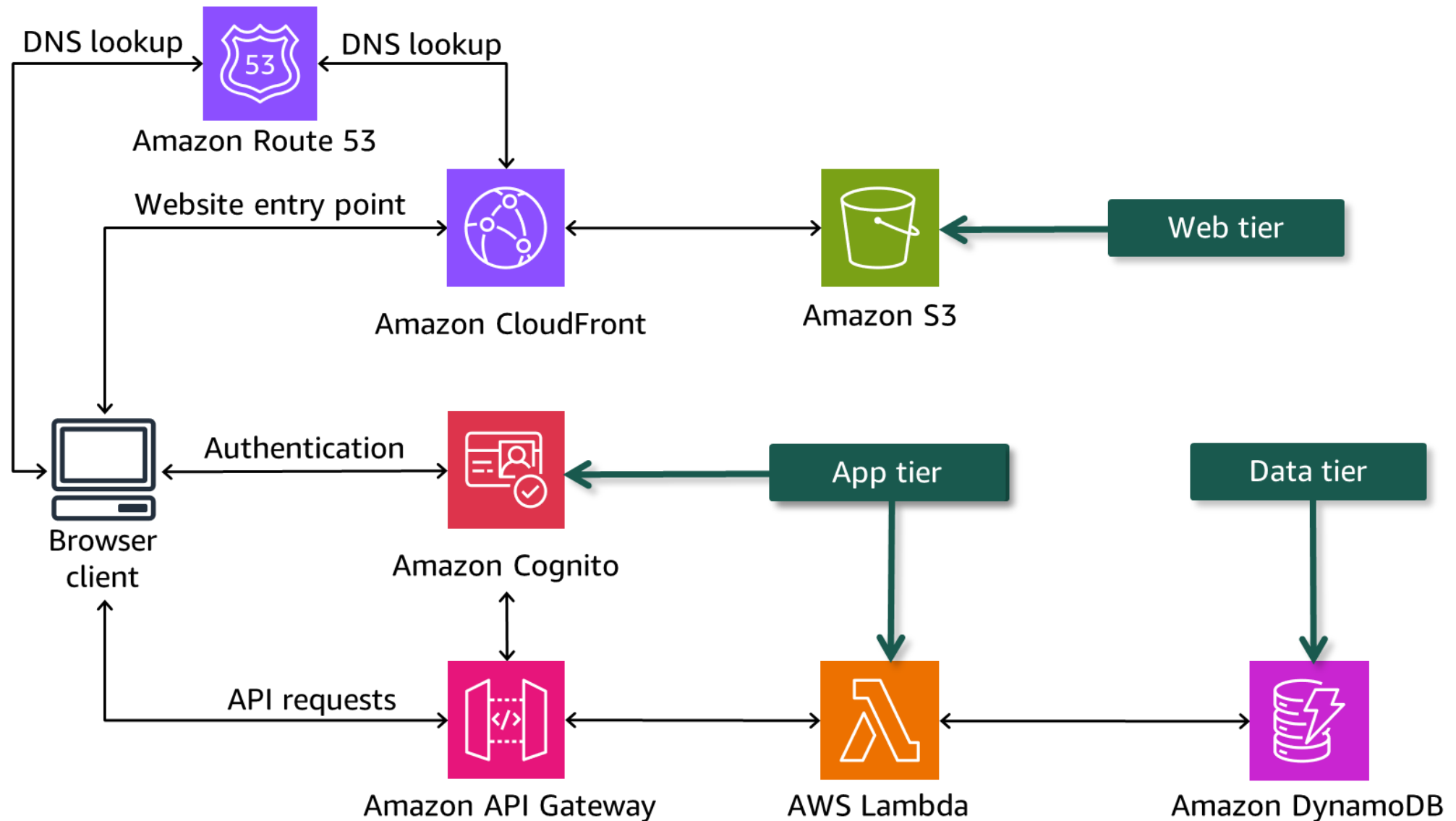


Amazon Redshift
Serverless



Amazon OpenSearch
Serverless

Web app three-tier design using AWS serverless services



Key takeaways: Thinking Serverless



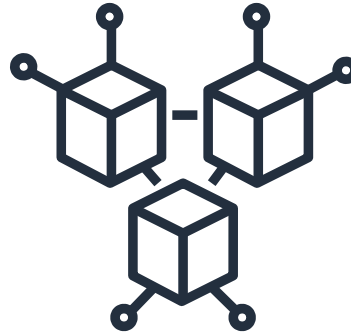
- AWS serverless services provide you the following benefits:
 - No server management
 - Pay-for-value services
 - Continuous scaling
 - Built-in fault tolerance
- Serverless services are suitable for event-driven and microservice architectures.



Architecting serverless microservices

Building Serverless Architectures and Microservices

Characteristics of a microservice



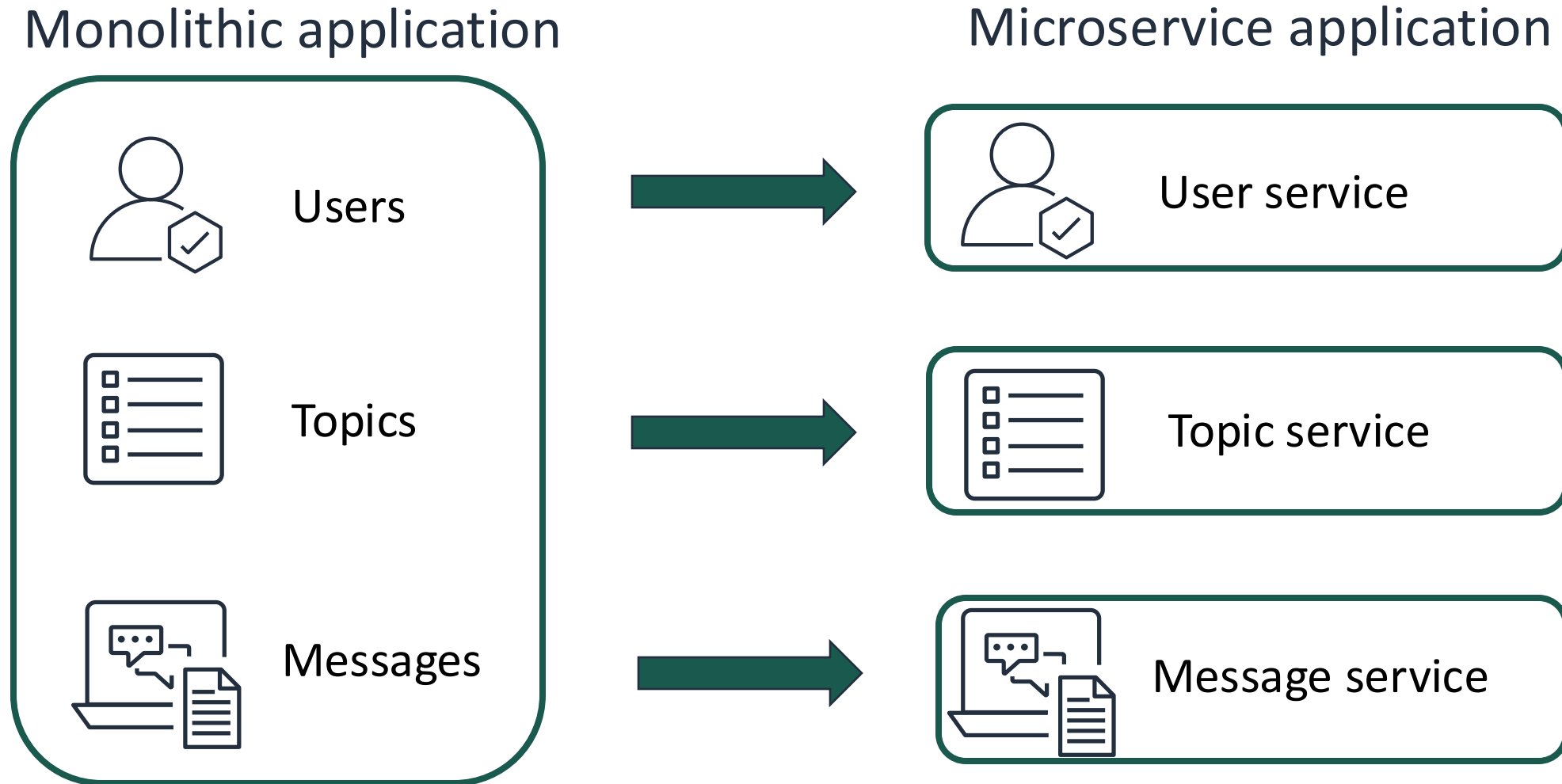
Autonomous

- Can be developed and deployed without affecting other microservices
- Scales independently
- Does not share code with other microservices
- Communicates over well-defined APIs

Specialized

- Performs a single business function solving a specific problem
- Is owned by a small development team that chooses development tools
- Is stateless
- Has its own data store

Example: Breaking a monolithic application into microservices



Benefits of microservices



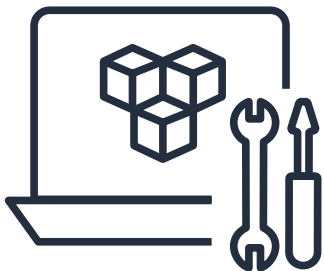
Team agility



Reusable code



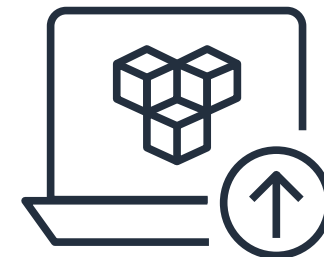
Flexible scaling



Technological freedom



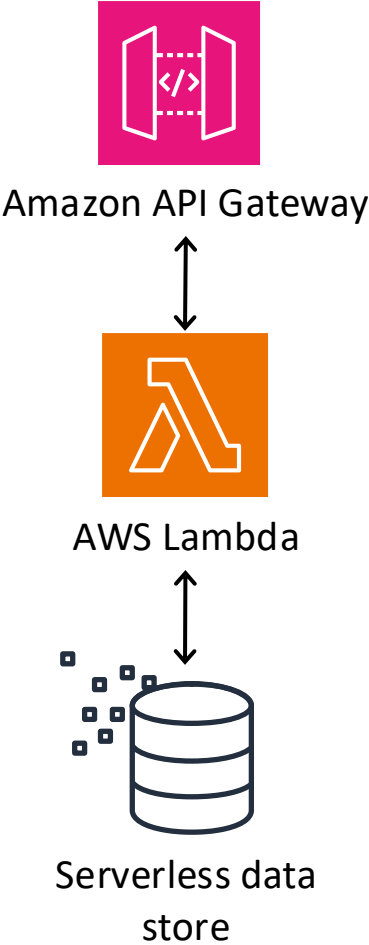
Resilience



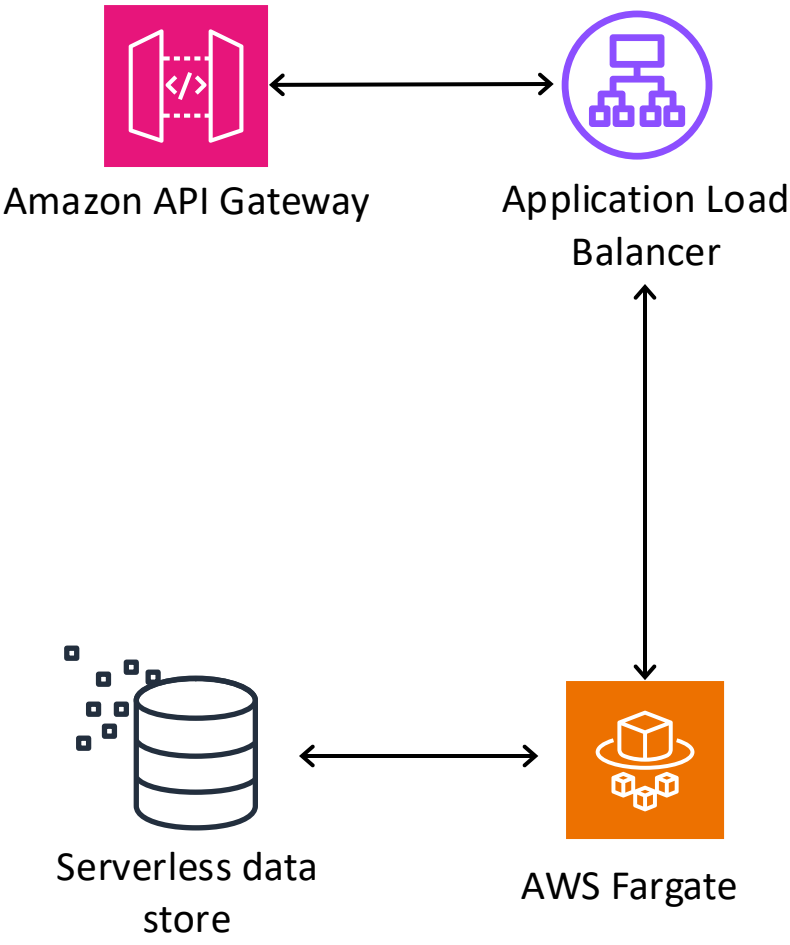
Simplified deployment

Microservice serverless patterns on AWS

RESTful APIs



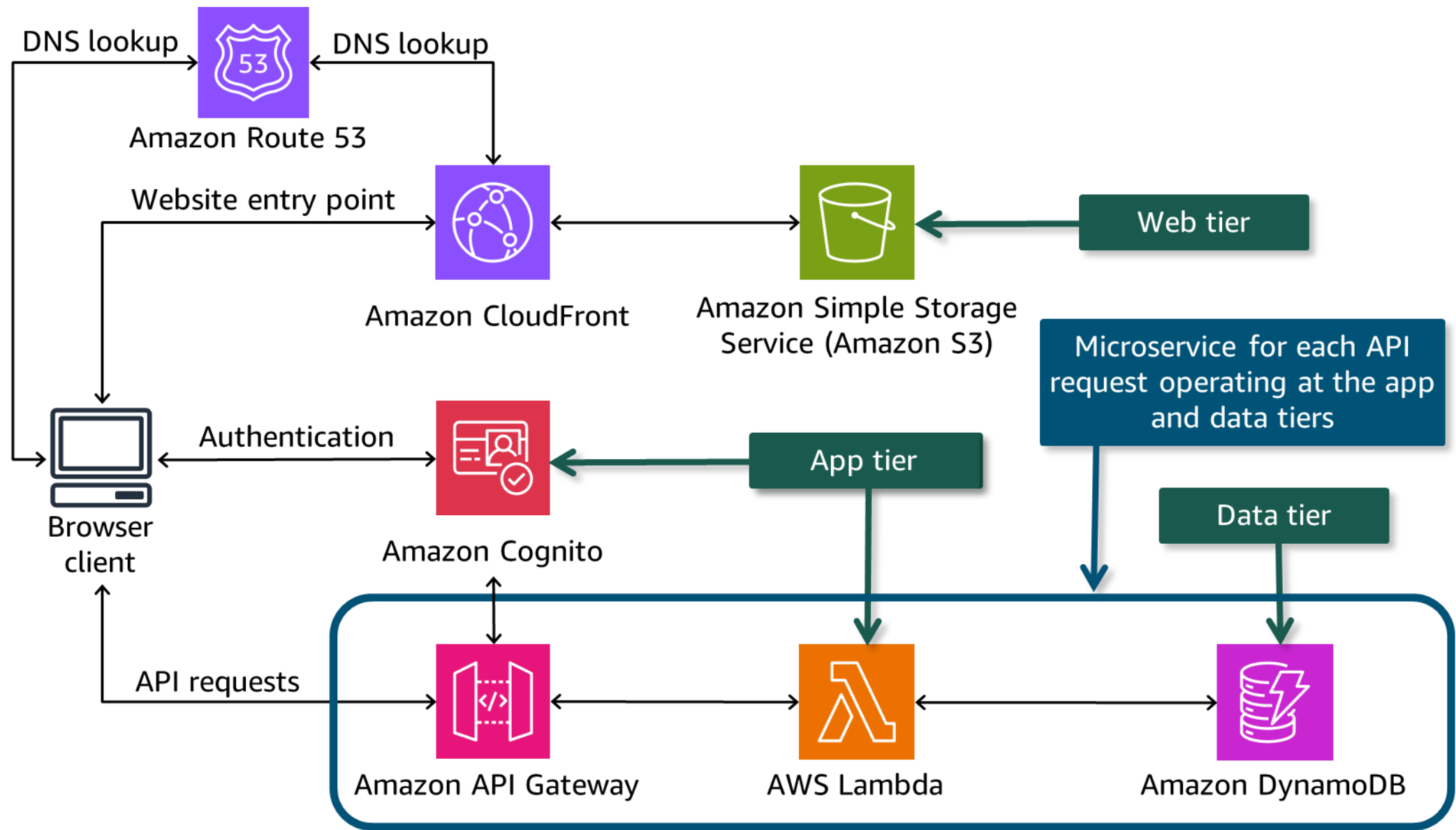
Containers



Streaming



Microservices in a web serverless three-tier architecture



Key takeaways: Architecting serverless microservices



- Microservice applications are composed of autonomous, specialized services.
- Microservices have the following benefits:
 - Team agility
 - Reusable code
 - Flexible scaling
 - Technological freedom
 - Resilience
 - Simplified deployment
- Microservices can form part of a three-tier architecture operating in the app and data tiers.



Building serverless architectures with AWS Lambda

Building Serverless Architectures and Microservices

Operational task comparison

Operational tasks	Server in a VPC	Serverless
Configure an instance	Yes	No
Update operating system (OS)	Yes	No
Install application platform	Yes	No
Build and deploy apps	Yes	Yes
Configure automatic scaling and load balancing	Yes	No
Continuously secure and monitor instances	Yes	No
Monitor and maintain apps	Yes	Yes

AWS Lambda

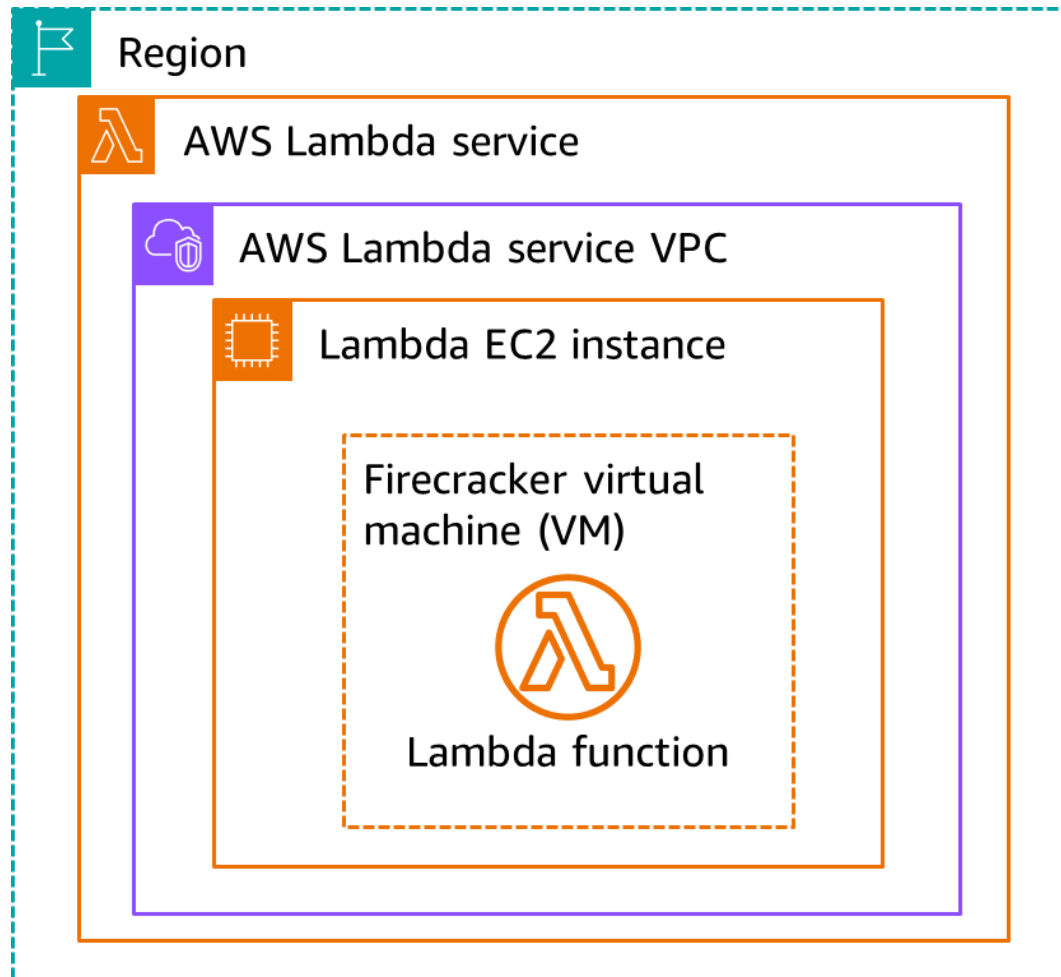


Lambda

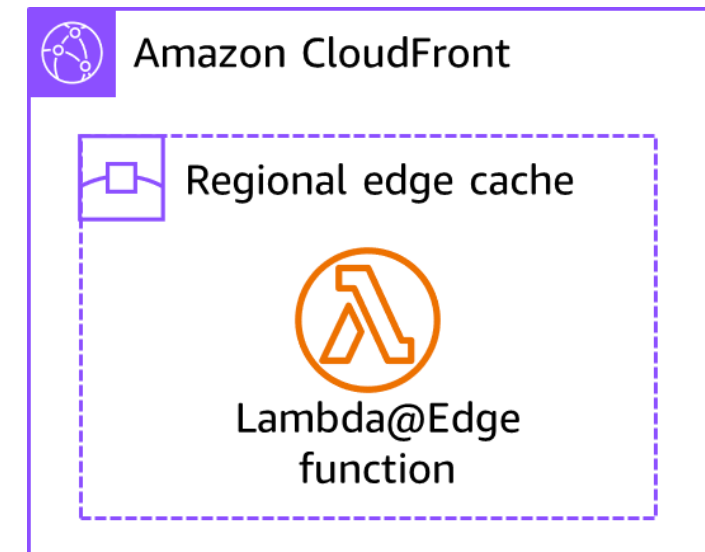
- AWS Lambda lets you run code functions without provisioning or managing servers.
- Lambda functions are configurable with runtime language, amount of memory, and timeout duration.
- A function has a maximum duration of 15 minutes.
- Functions can be deployed as .zip files or container images.

Lambda function location options

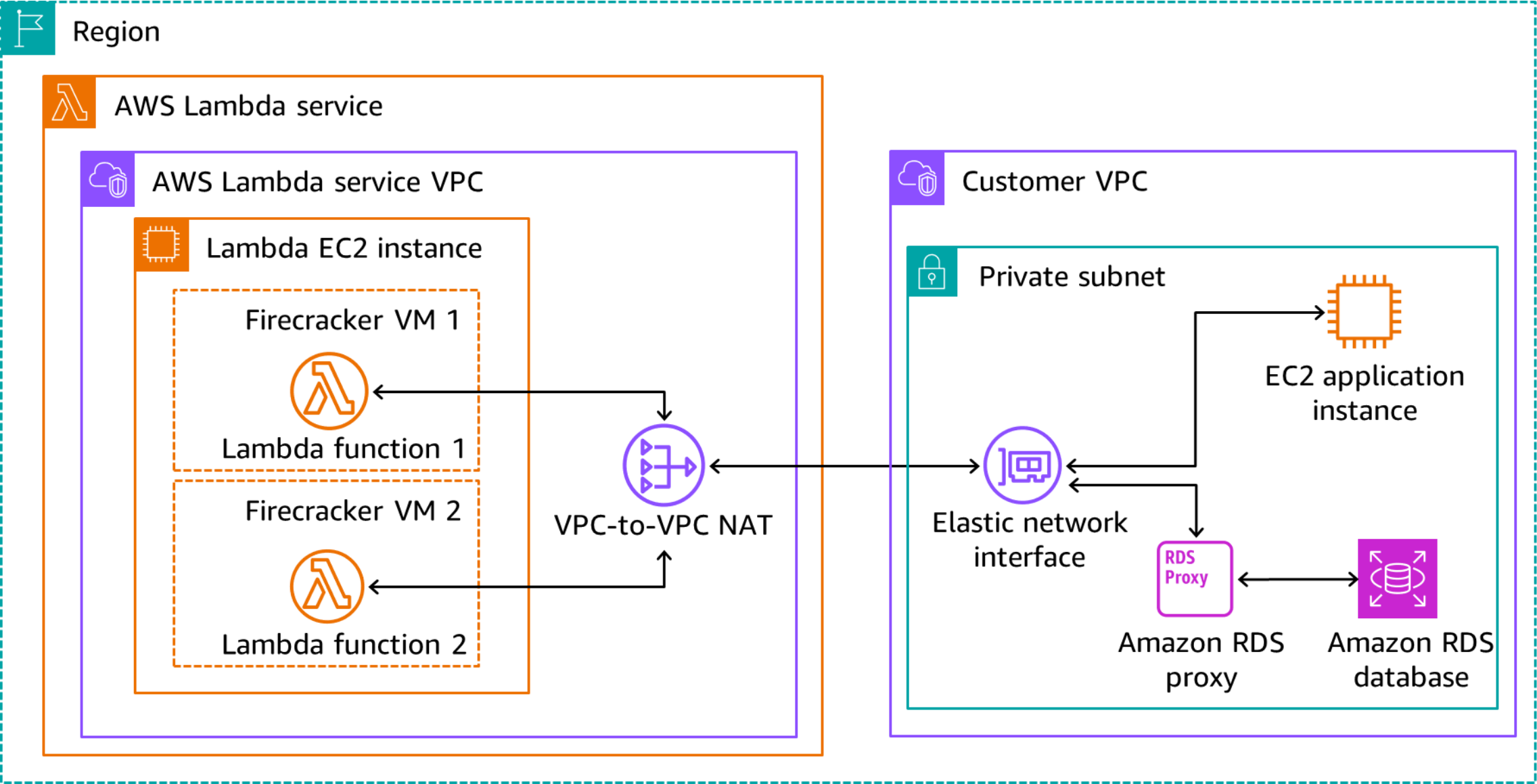
In the AWS Lambda service



In an Amazon CloudFront regional edge cache



Connecting a Lambda function to your VPC



Identifying Lambda serverless scenarios



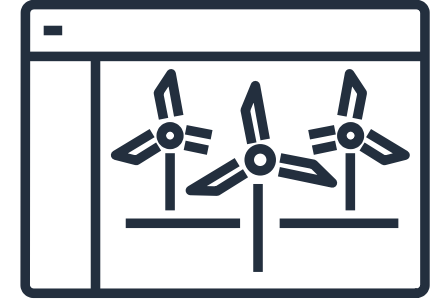
Synchronous processing

- Web apps
- Web services
- Microservices
- Machine learning inferences



Asynchronous processing

- Scheduled events
- Queued messages
- Image or video transformation
- AWS service triggers

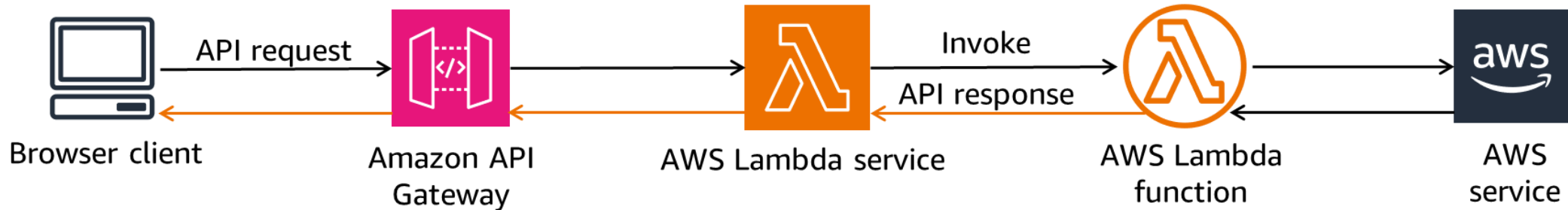


Streaming processing

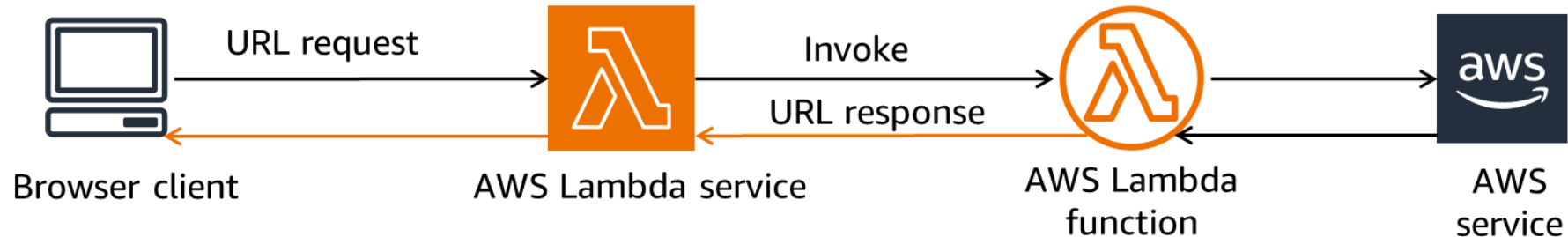
Streaming applications

Invoking a synchronous Lambda function

External API request using API Gateway

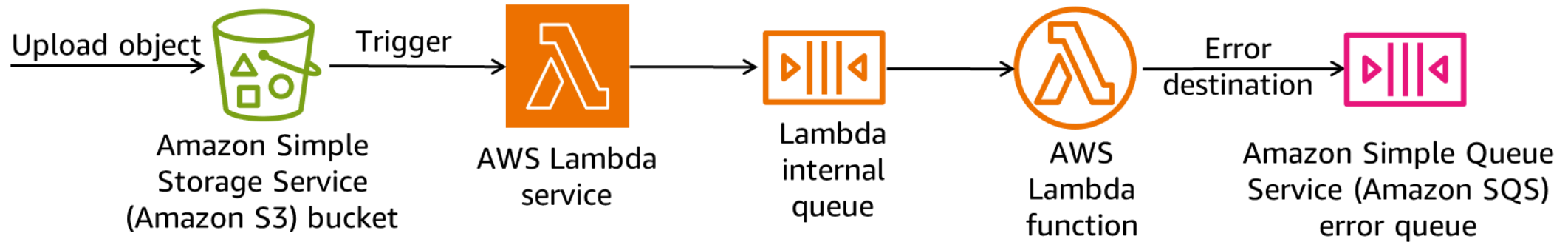


External API request using Lambda function URL

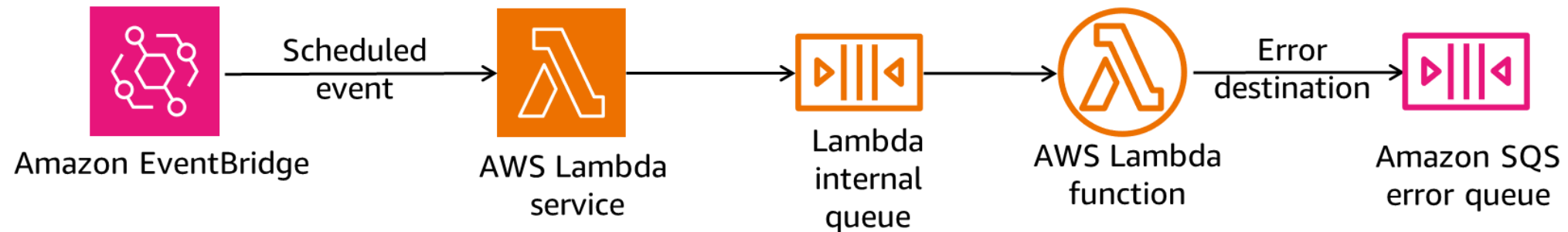


Invoking an asynchronous Lambda function

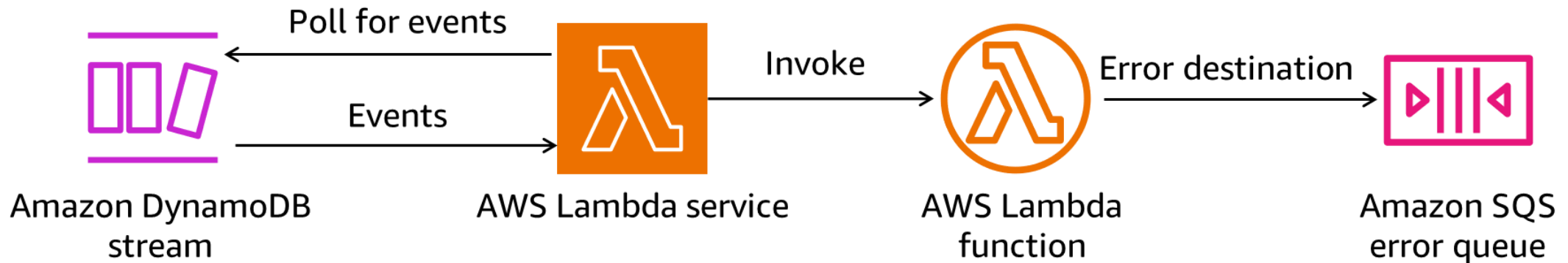
AWS service trigger event



Scheduled event



Lambda event source mappings for queues and streams



Python Lambda function handler

```
1 import json
```

Line 2: Event object is a JSON document containing input data and invoking service data

```
2 def lambda_handler(event, context):
```

Line 2: Context object provides methods and properties about function runtime and invocation

```
3     length=event['length']
```

```
4     width=event['width']
```

```
5     area = calculate_area(length, width)
```

```
6     data = {"area": area}
```

```
7     return json.dumps(data)
```

Line 7: json.dumps(data) returns result as a JSON document

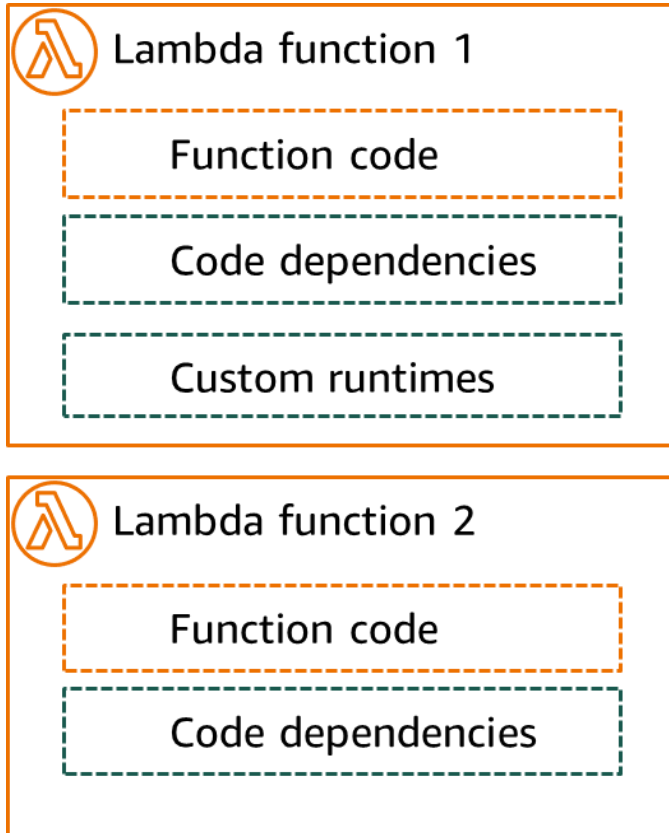
```
8 def calculate_area(length, width):
```

```
9     return length*width
```

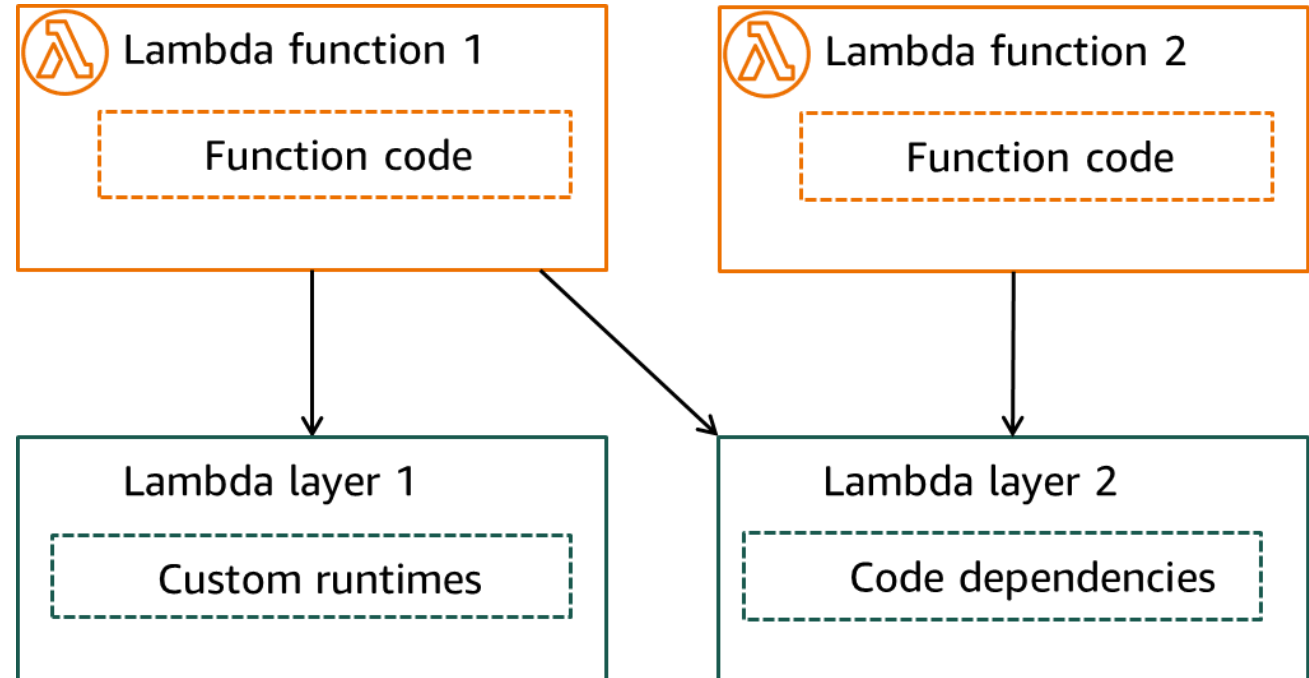
Lines 8 and 9: Business logic method

Lambda layers

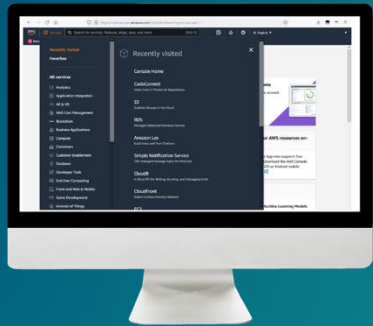
Without layers



With layers



Demo: Using AWS Lambda with Amazon S3



- This demonstration uses AWS Lambda and Amazon S3.
- In this demonstration, you will see how to do the following:
 - Use AWS Lambda with Amazon S3 to upload an object.
 - Create a thumbnail for the object.

Key takeaways: Building serverless architectures with AWS Lambda



- AWS Lambda is a service to run code functions without provisioning or managing servers.
- A Lambda function can run inside a VPC owned by the AWS Lambda service or as Lambda@Edge in an Amazon CloudFront regional cache.
- A Lambda function can be configured to connect to your VPC to access AWS services inside the VPC.
- A Lambda function can be invoked synchronously, asynchronously, and with event source mappings for queues and streams.
- Use Lambda layers to package code dependencies or custom runtimes to be re-used by all Lambda functions in the Region.



Guided lab: Implementing a Serverless Architecture on AWS (Lambda lab)

Lambda lab tasks:



- In this lab, you will do the following:
 - Implement a serverless architecture on AWS.
 - Trigger Lambda functions from Amazon S3 and Amazon DynamoDB.
 - Configure Amazon Simple Notification Service (Amazon SNS) to send notifications.
- Open your lab environment to start the lab and find additional details about the tasks that you will perform.

Debrief: Lambda lab

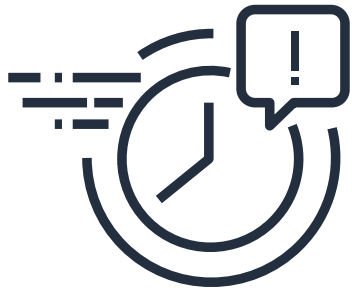
- When you created your Load-Inventory function, you gave the function a role. What does a role do for a function?
- In this lab, you created multiple Lambda functions. What is the benefit of using multiple functions rather than combining multiple tasks into a single function?



Building microservice applications with AWS container services

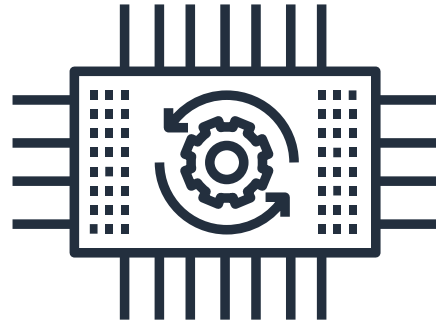
Building Serverless Architectures and Microservices

Choosing containers over AWS Lambda functions



Longer than 15 minutes

If a service runs longer than 15 minutes, it isn't suitable for Lambda functions.



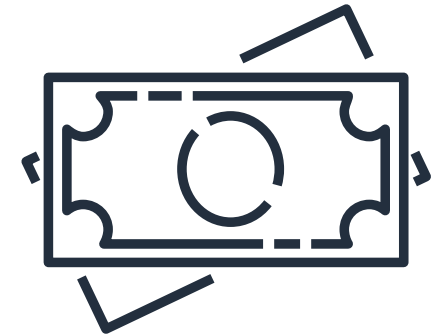
Memory intensive applications

Workloads that use more than 10 GB of memory are not suitable for Lambda functions.



Migrating legacy containers

Containers can help migrate legacy applications running on-premises or on EC2 instances without refactoring or rearchitecting them.

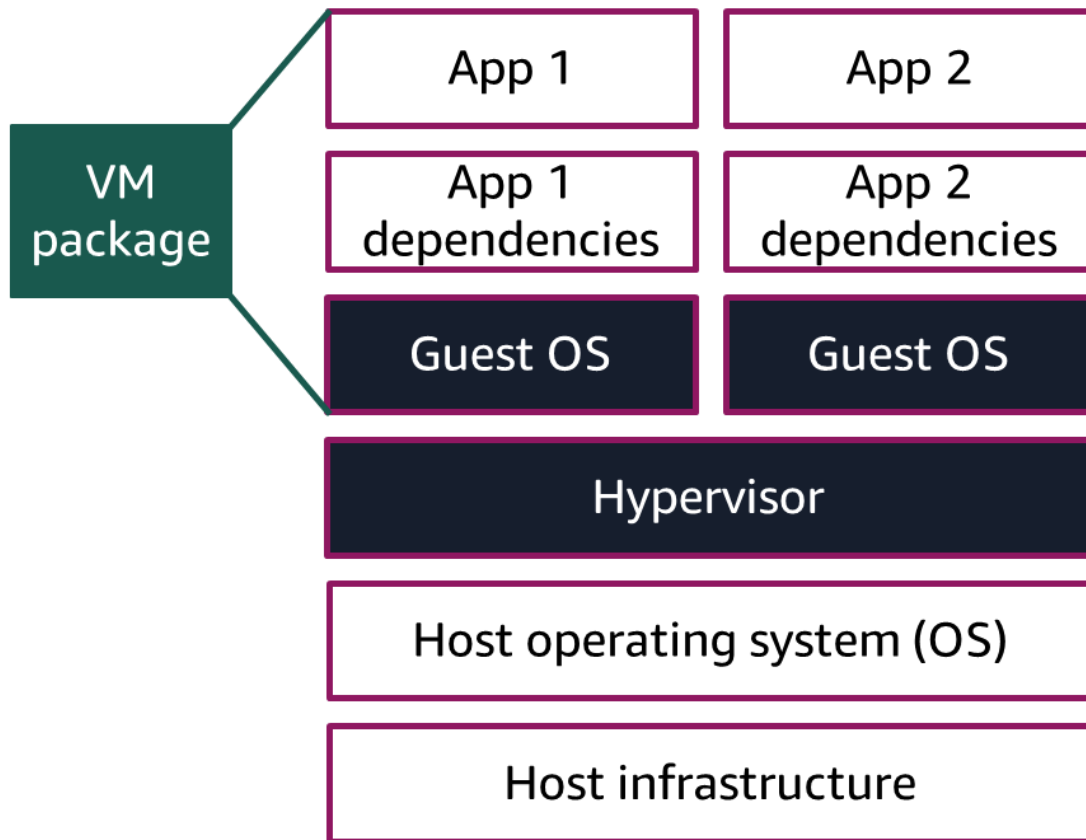


Cost

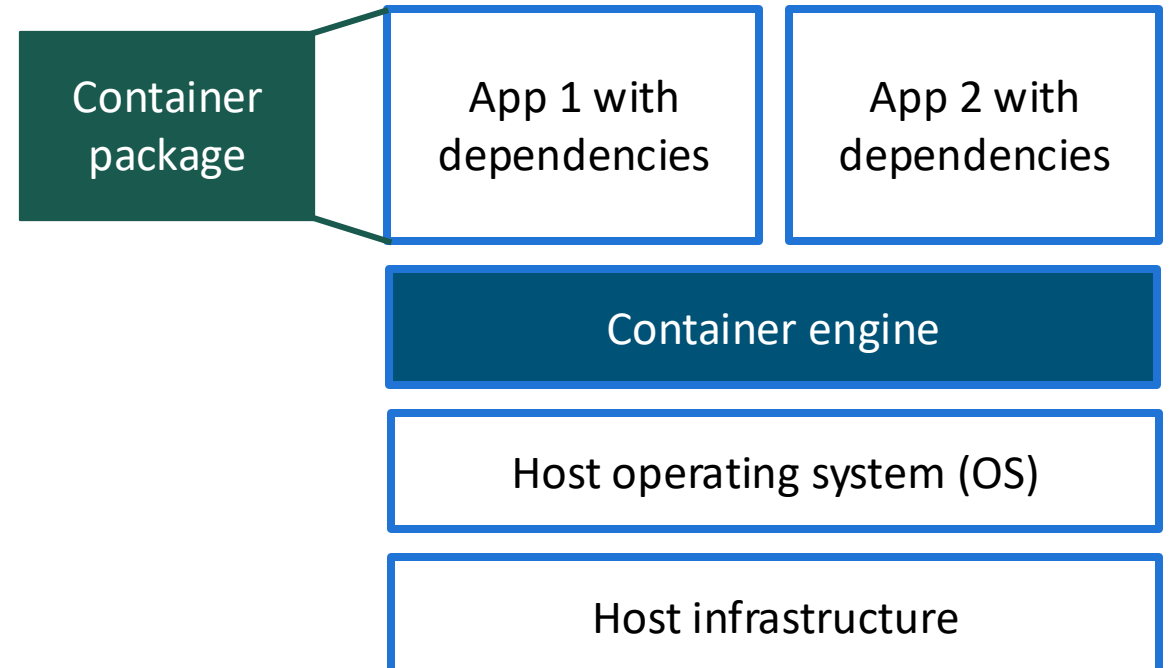
- Containers can be continuously run at a fixed cost.
- The price to run a Lambda function increases with the number of invocations, duration, and memory allocated.

The benefit of containers

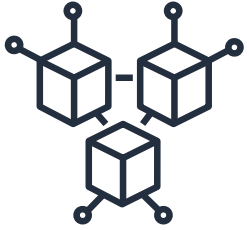
Virtual machines



Containers



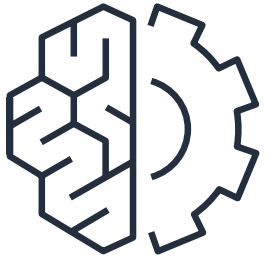
Container use cases



Microservice applications



Batch processing



Scale machine learning (ML) models

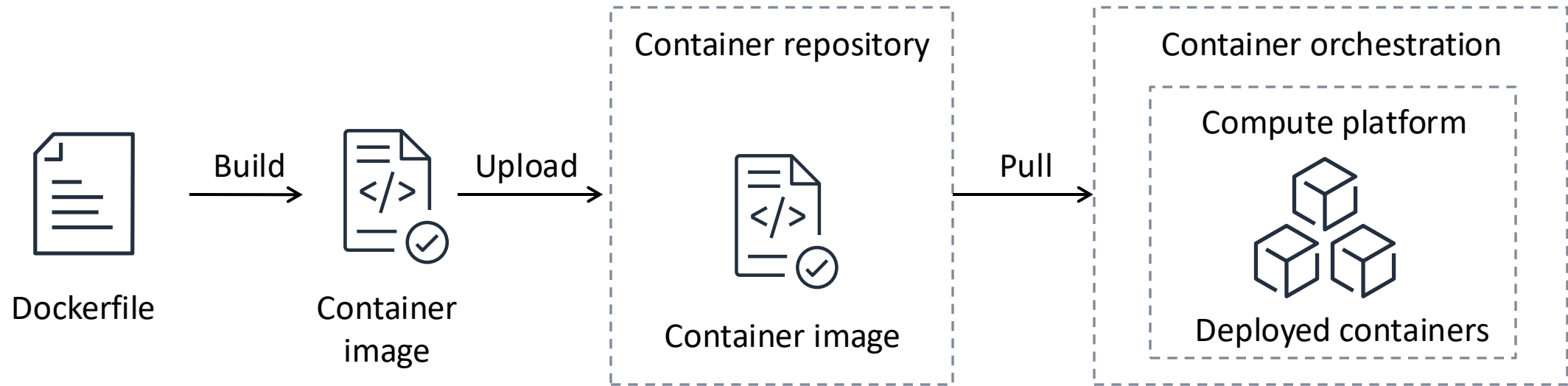


Standardize hybrid architecture applications



Migrate applications to the cloud

Creating and deploying Docker containers



AWS container services

Registry



Amazon Elastic Container
Registry (Amazon ECR)

Orchestration

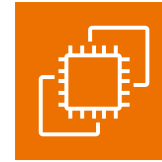


Amazon Elastic
Container Service
(Amazon ECS)



Amazon Elastic
Kubernetes Service
(Amazon EKS)

Compute options



Amazon Elastic Compute
Cloud (Amazon EC2)

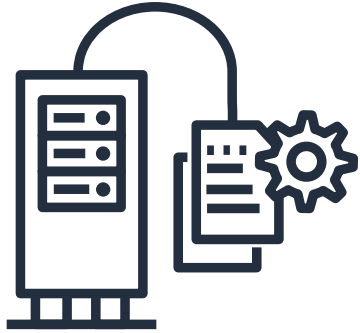


AWS Fargate



AWS Lambda

Benefits of AWS Fargate



No cluster or server management

- No server provisioning or maintenance.
- No cluster packing optimization.



Per second billing

Only pay for what you use.



Automatic scaling

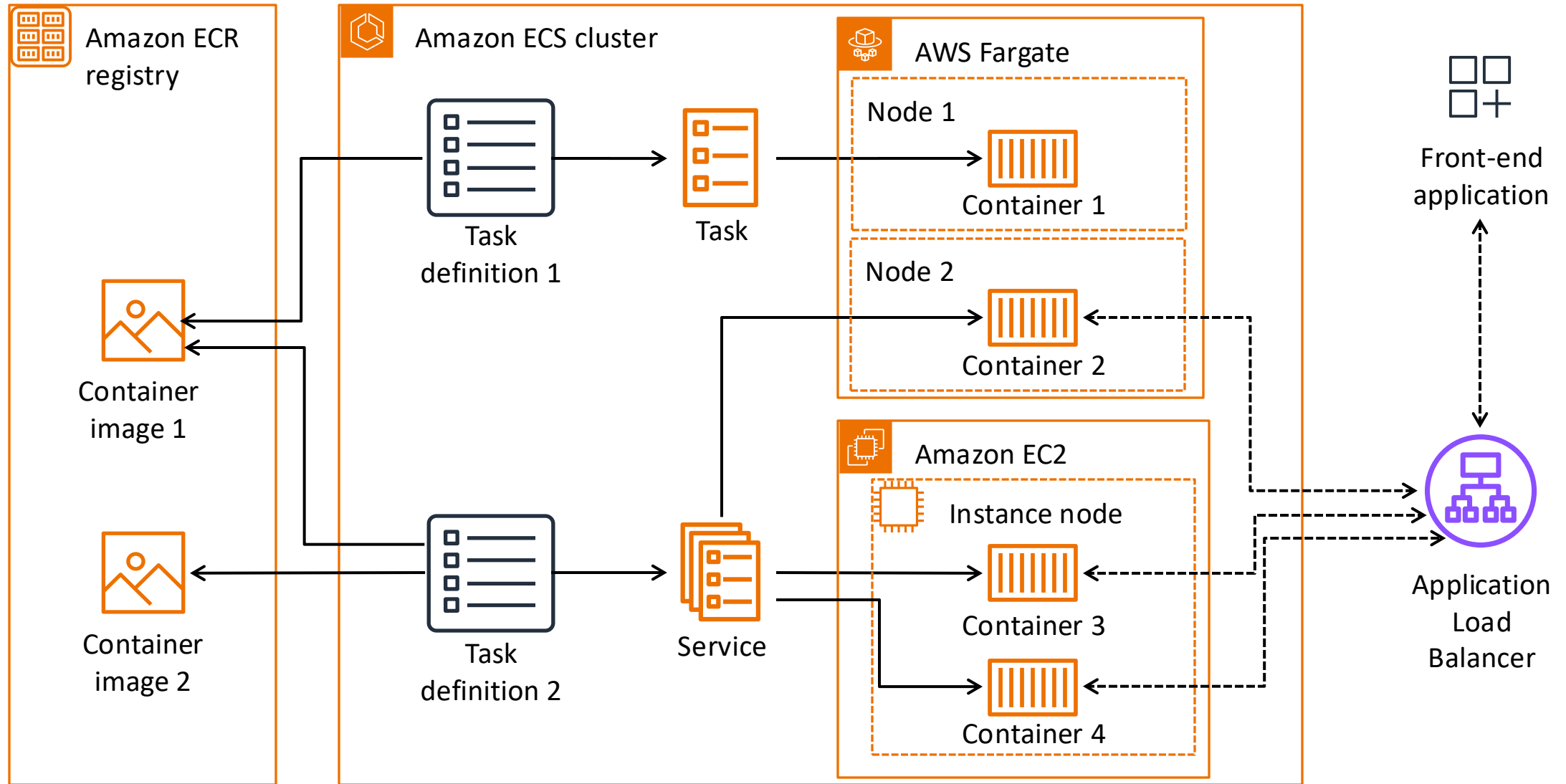
Dynamically scale tasks based on CPU, memory, or other metrics.



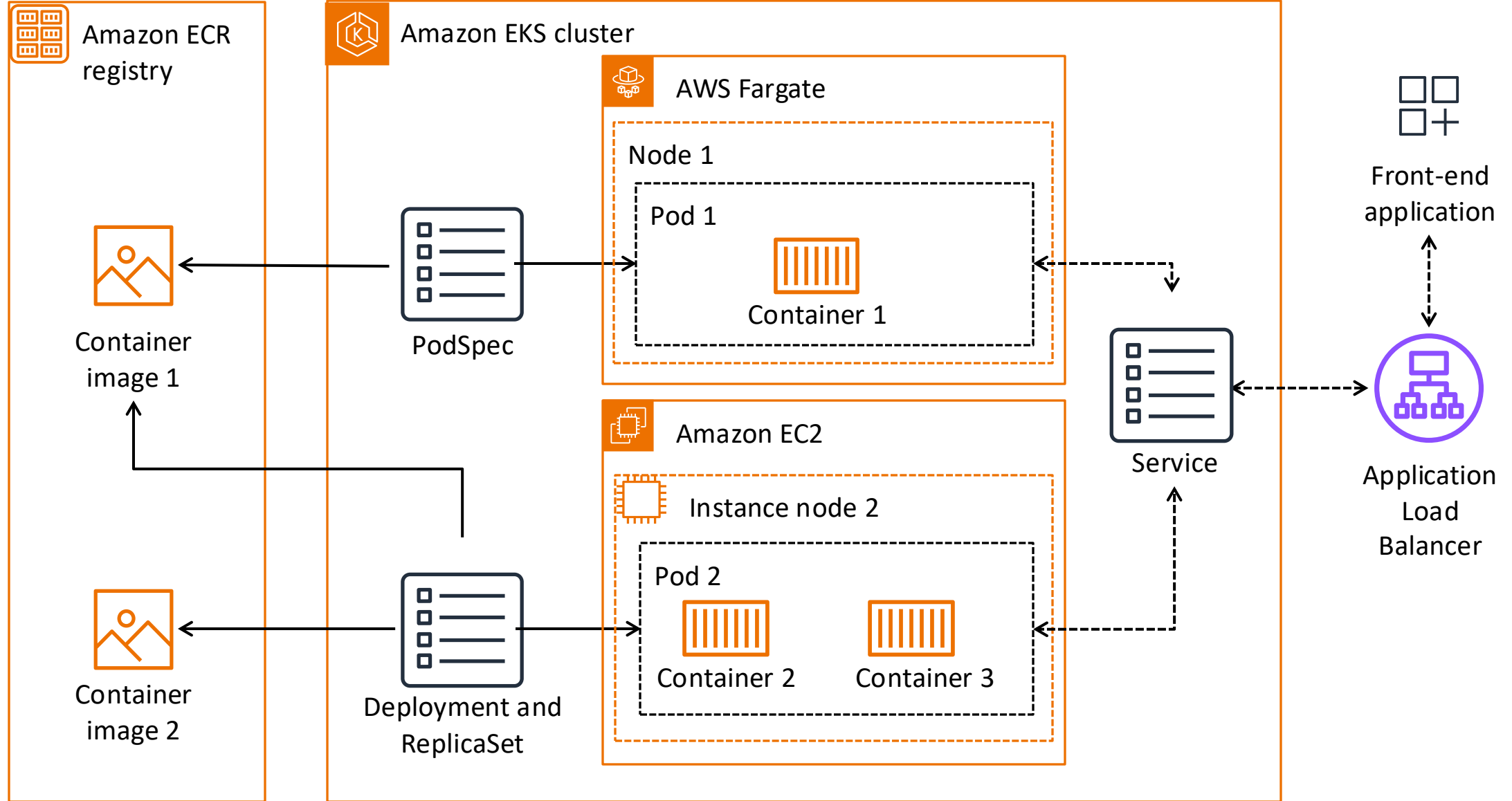
Suitable for teams new to containers

AWS Fargate usage doesn't require comprehensive knowledge of container technology.

Deploying and invoking containers on Amazon ECS



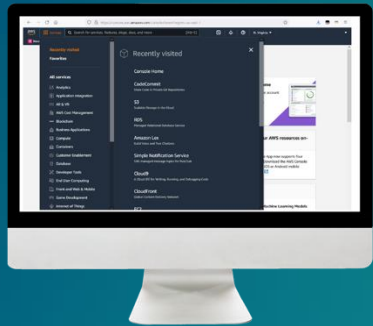
Deploying and invoking containers on Amazon EKS



Choosing between Amazon ECS and Amazon EKS

Topic	Amazon ECS	Amazon EKS
Complexity	Simplifies cluster creation and maintenance	Provides more control over cluster using a complex interface
Scaling	Automated scaling based on demand	Manual configuration of autoscaling groups
Toolset	Amazon ECS toolset	Kubernetes toolset
Team experience	New to container cluster architecture	Familiar with Kubernetes cluster architecture and control processes

Demo: Running a Container



- This demonstration uses Amazon Elastic Container Service.
- In this demonstration, you will see how to do the following:
 - Run a sample application on an Amazon ECS cluster behind a load balancer.

Key takeaways: Building microservice applications with AWS container services



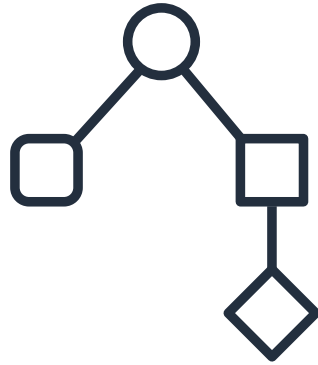
- Choose a container solution instead of Lambda functions when an application requires resources exceeding Lambda service limits.
- Amazon ECR provides a container image repository service.
- Amazon ECS is a container orchestration service with AWS toolkit.
- Amazon EKS is a container orchestration service with Kubernetes toolkit.
- AWS Fargate manages serverless cluster nodes and can be deployed in Amazon ECS and Amazon EKS clusters.



Orchestrating microservices with AWS Step Functions

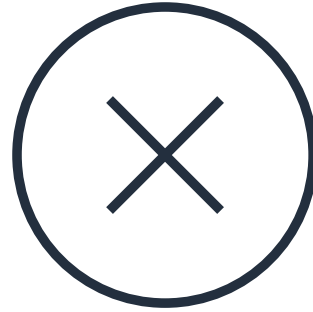
Building Serverless Architectures and Microservices

Challenges with microservice applications



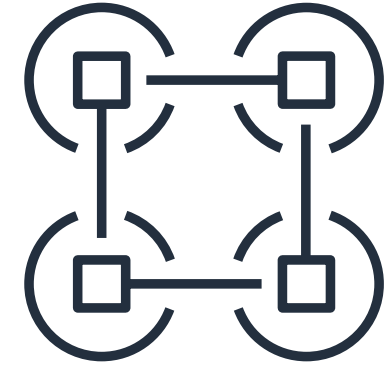
Dependencies

- Handling microservice dependencies
- Chaining microservices sequentially or in parallel



Error scenarios

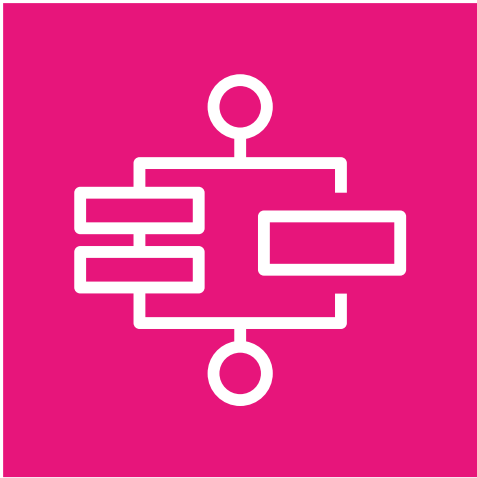
Handling retries after a timeout or error



Microservice coordination

- Scaling an application
- Maintaining state for stateless microservices
- Passing data between microservices
- Monitoring and troubleshooting

AWS Step Functions



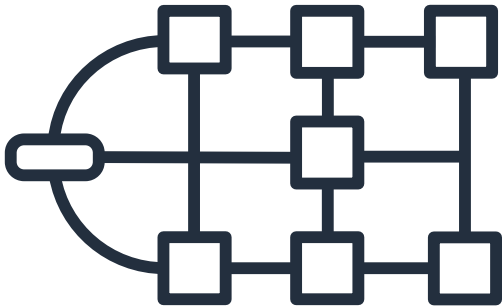
Step
Functions

- Is a serverless orchestration service to manage workflows between multiple AWS services
- Has state machines (workflows) that contain a series of event-driven states (steps)
- Manages each workflow's state, checkpoints, and restarts
- Provides error handling capability
- Can transfer data between states
- States can filter and manipulate data

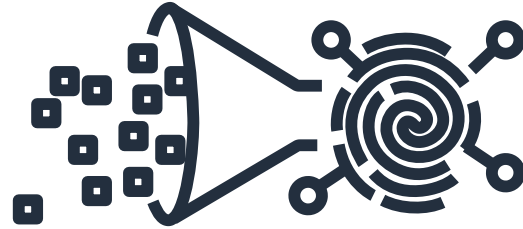
Standard or Express Workflow types

Topic	Standard Workflows	Express Workflows
Duration	Long-running	Short-running with no activities
Run metrics	Full history in the console	Results in CloudWatch Logs
Processing	Asynchronous	Synchronous and asynchronous
Run model	Exactly-once	Synchronous: At-least-once Asynchronous: At-most-once
State machine progress	Persisted on every state transition	No persisted state on every state transition
Pricing	Charged based on the number of state transitions	Charged based on the number of requests for workflow and its duration

Step Functions use cases



Microservice
orchestrations



Data processing

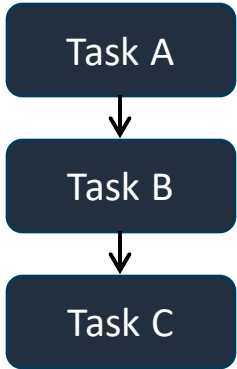


Machine learning (ML)

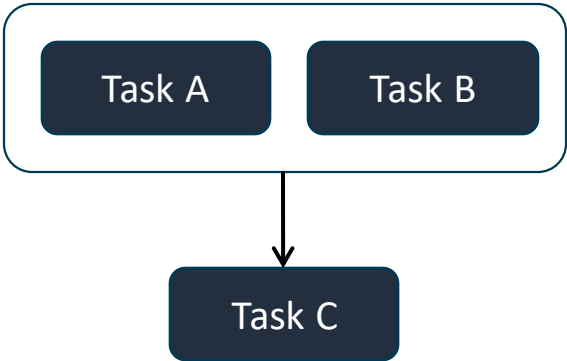


Security automation

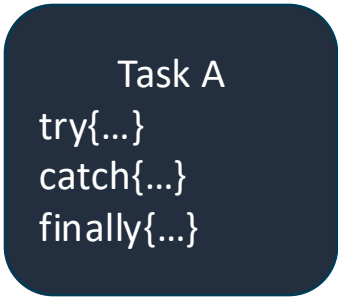
Workflow state coordination



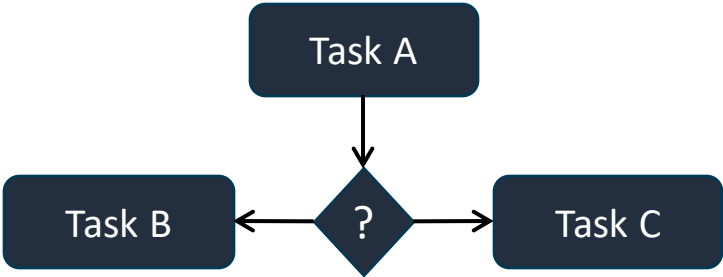
Run tasks in sequence



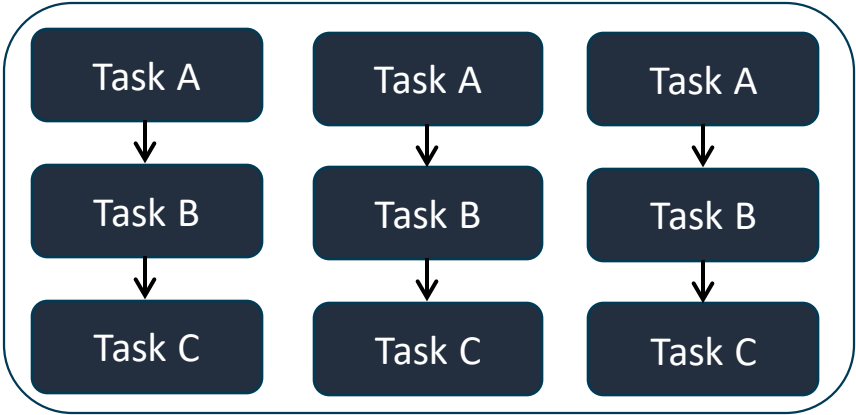
Run tasks in parallel



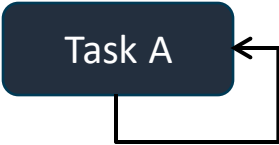
Manage errors with try-catch-finally



Select task based on data



Process data record tasks in parallel



Retry failed tasks

State machine state (step) types

Work states

- **Task:** Integrates with AWS services
- **Activity:** Performs a task hosted anywhere
- **Pass:** Passes or filters input data to next state
- **Wait:** Delays workflow for a specified time
- **State has wait for callback state option:** Pauses workflow and waits for callback

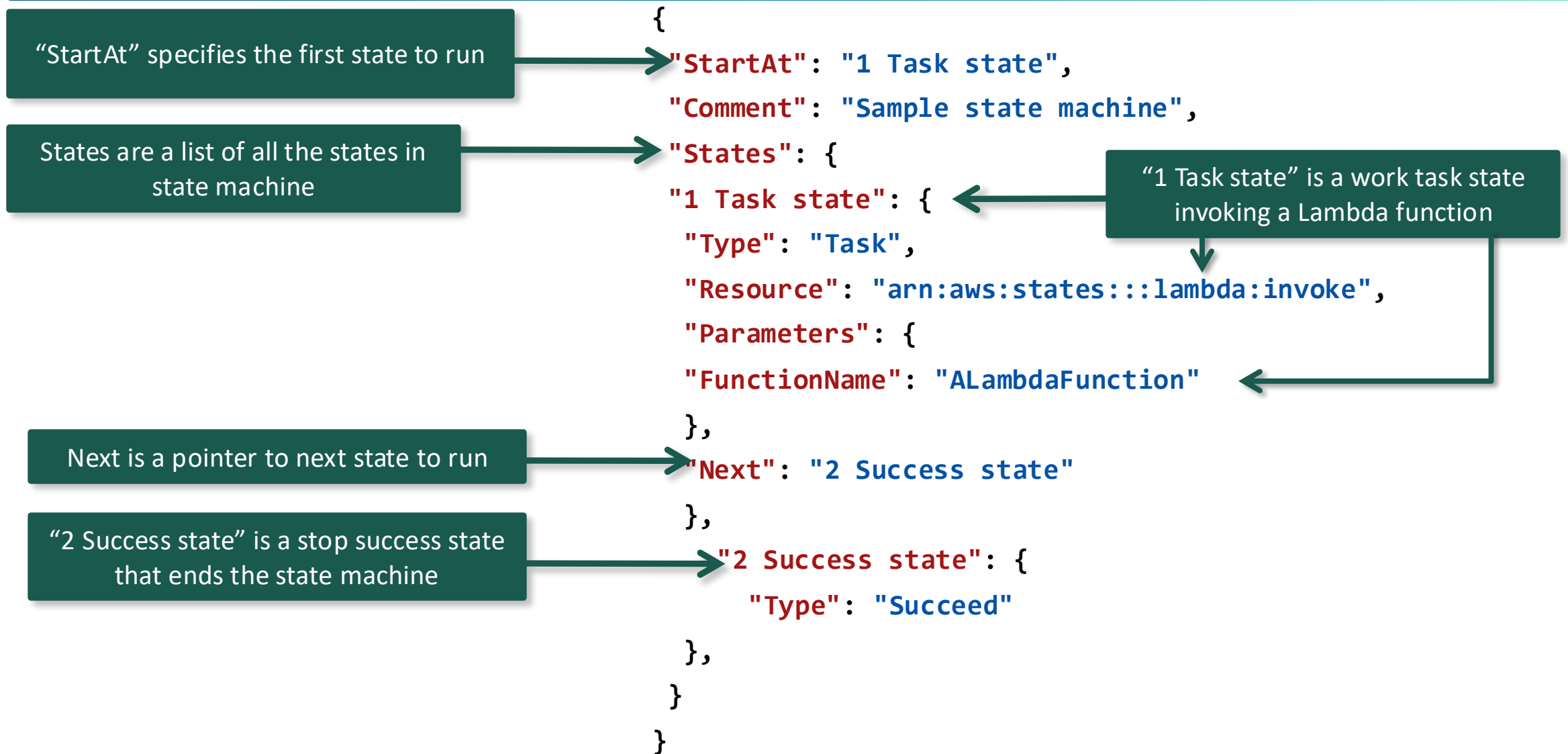
Transition states

- **Choice:** Adds conditions to control flow to next state
- **Parallel:** Adds branches of nested state machines inside a state machine
- **Map:** Separates workflow for each data record in data set that runs in parallel

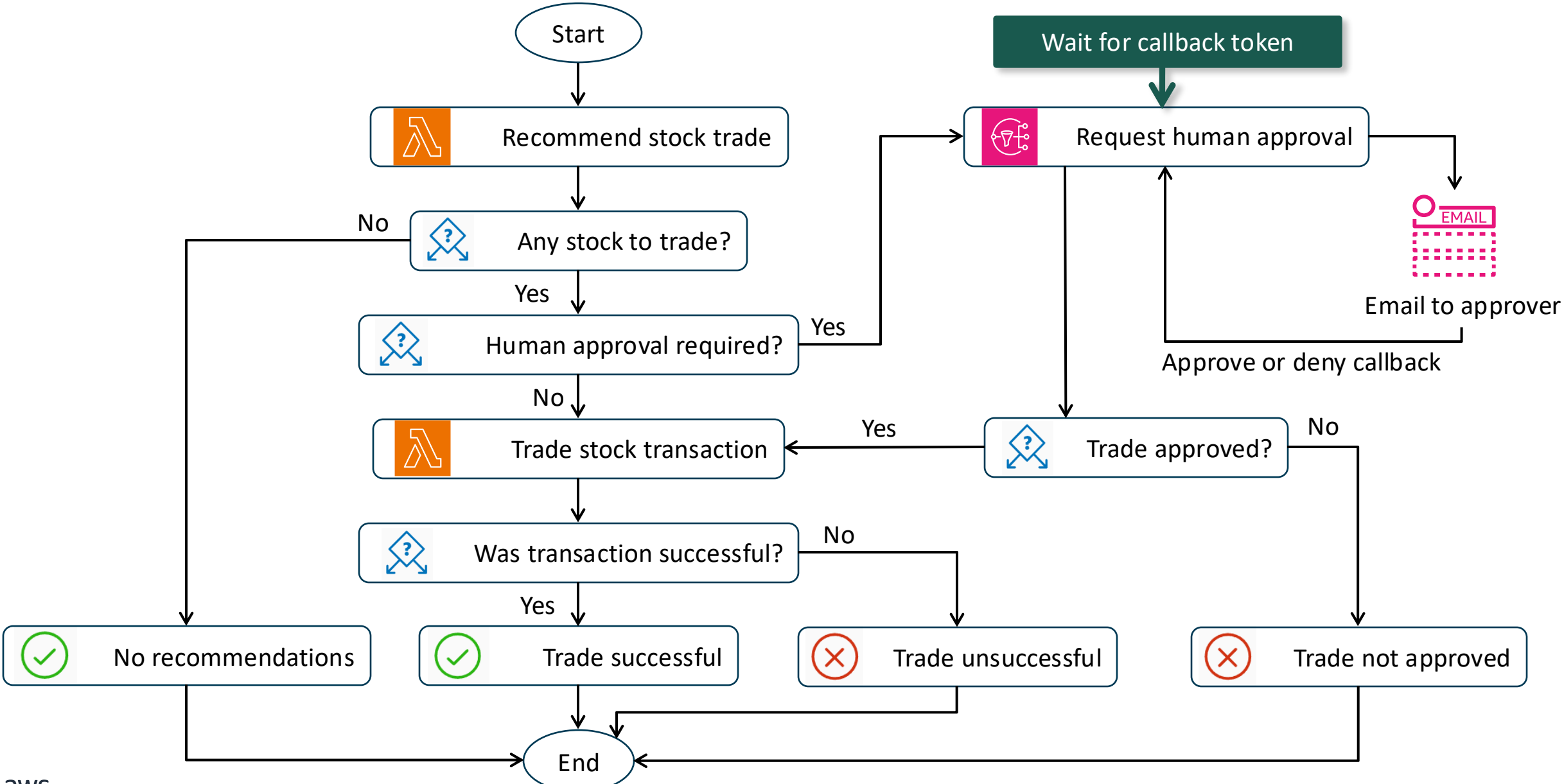
Stop states

- **Success:** Stops state machine and marks execution as successful
- **Fail:** Stops state machine and marks execution as failed
- **State has the end parameter:** Stops state machine

Defining a state machine with Amazon State Language



Example: Step Functions Standard Workflow state machine



Key takeaways: Orchestrating microservice workflows with AWS Step Functions



- AWS Step Functions is a serverless orchestration service to manage workflows between multiple AWS services.
- A state machine (workflow) is a series of event-driven states (steps).
- A Step Functions state machine is a collection of states defined in the Amazon States Language.
- States can be grouped into work, transition, and stop states.
- A task state can invoke an AWS service or request an activity hosted on any compute service with an HTTP connection.



Extending serverless architectures with Amazon API Gateway

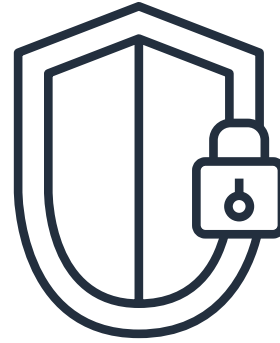
Building Serverless Architectures and Microservices

Benefits of APIs



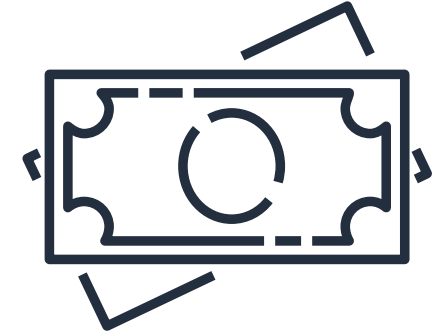
Standardize app communication

- Connect software apps written in different languages in a standardized way.
- Hide implementation complexity.



Protect microservices

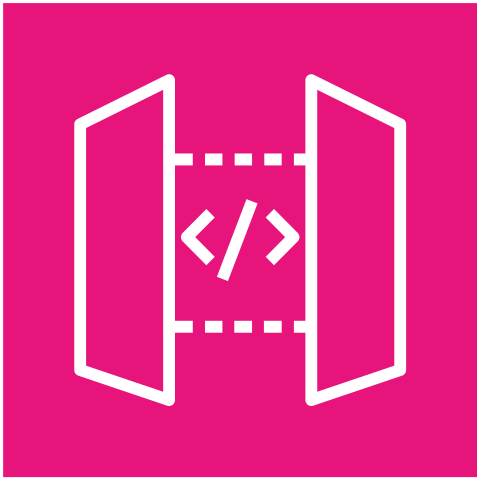
- Choose whether to require authorization.
- Check request formats.
- Throttle the number of requests.
- Restrict resource access.



Monetize API and track statistics

- Track client usage for billing purposes.
- Supply usage statistics per client.

Amazon API Gateway



API
Gateway

- Provides ability to create, publish, and maintain REST, HTTP, and WebSocket APIs
- Configurable traffic management, authorization, and resource access control
- Provides access to AWS services and publicly accessible endpoints
- Hosts multiple versions and stages of an application's API
- Establishes client usage plans to monetize and control APIs
- Can cache common responses

Choosing an API type

REST APIs

- Collection of routes and methods
- For apps that require API management features, such as usage plans, payload validation, private API endpoints, and resource policies
- Supports cross-origin resource sharing (CORS)
- Stateless

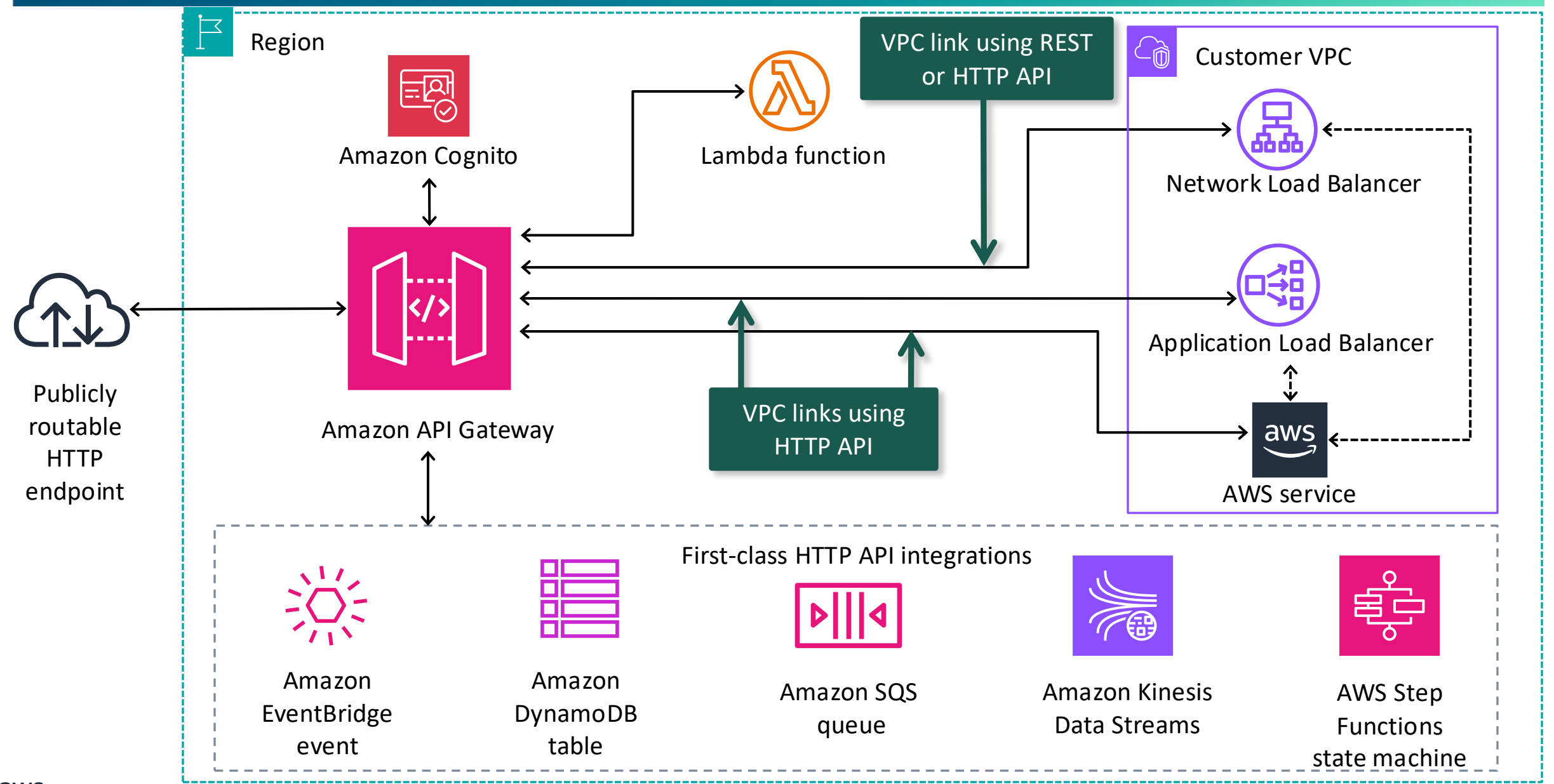
HTTP APIs

- Collection of routes and methods
- For microservices
- Lower latency and lower cost than REST APIs
- Supports CORS
- Stateless

WebSocket APIs

- Collection of WebSocket routes
- For real-time applications
- Establishes a session between client and backend services
- API management features are validate payload schema and data transformations
- Stateful

API Gateway backend integrations



Activity: Decomposing a Monolithic Application with AWS API Gateway



Choose the appropriate Amazon API Gateway API and AWS service or services to decompose an online shopping monolithic application into services. The following are the components of the monolithic shopping application:

- Shopping cart
- Payments
- Delivery

Decomposing a monolithic application into microservices or workflows with API Gateway

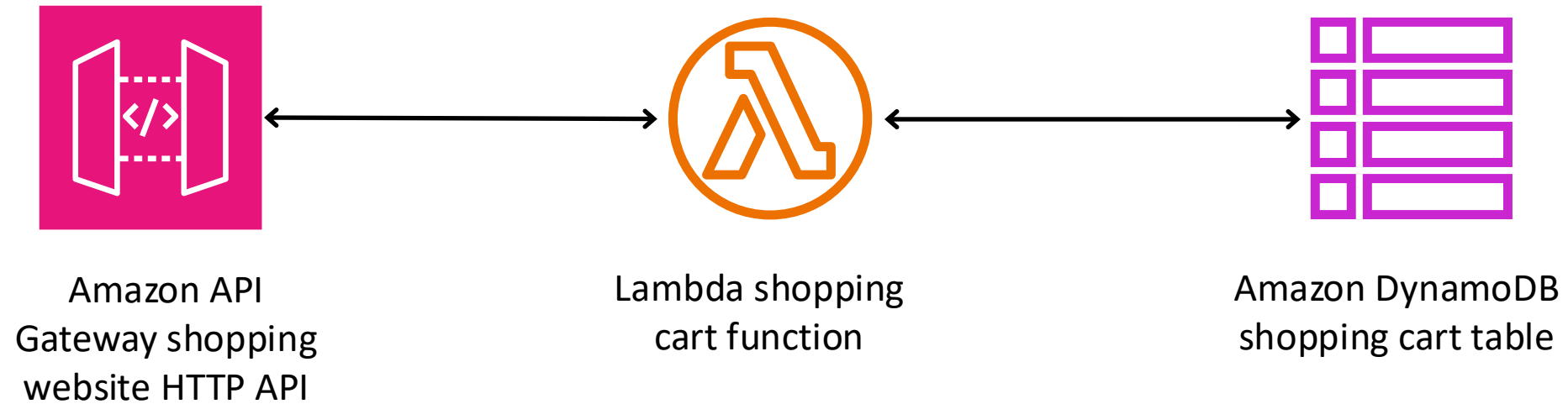
Monolithic application



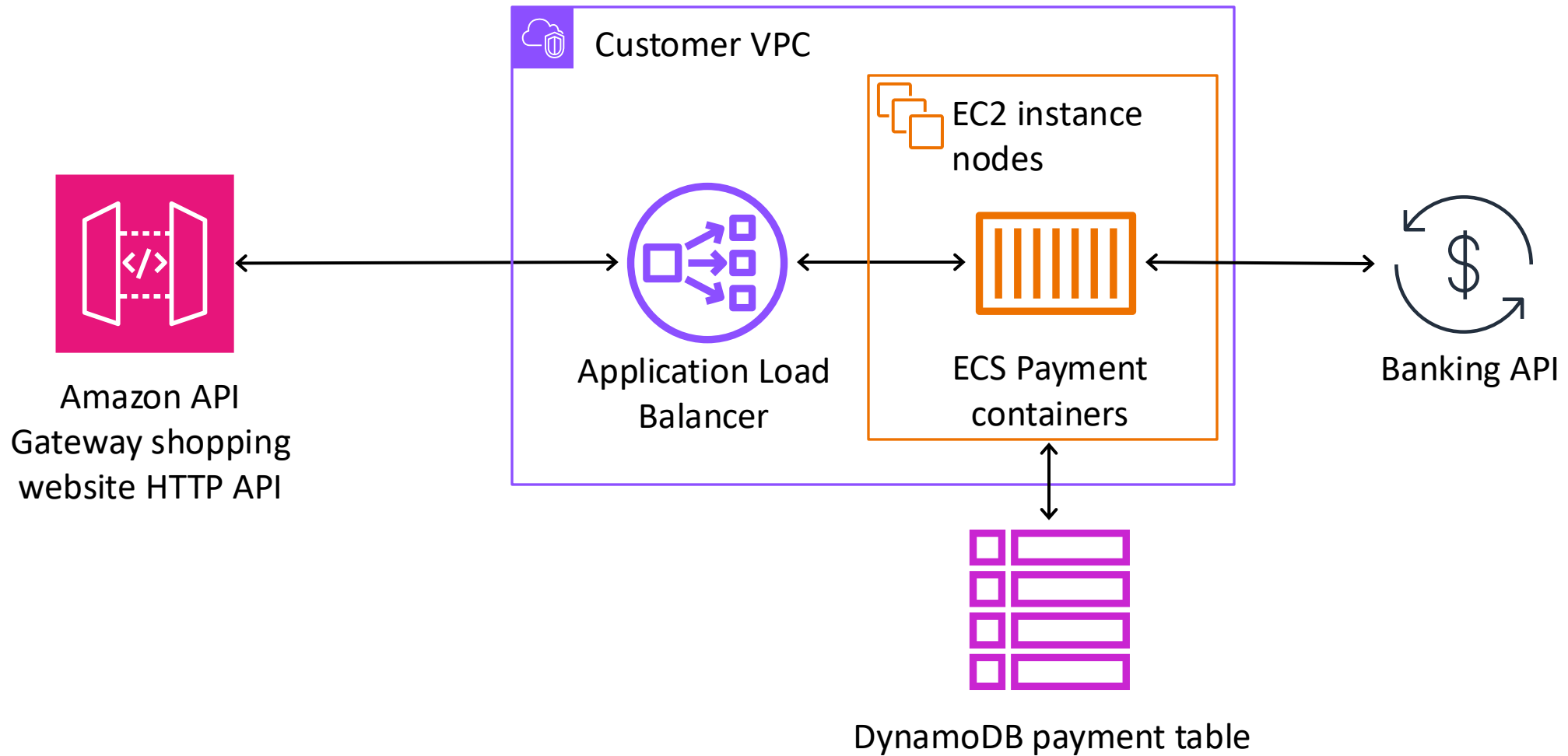
Service applications



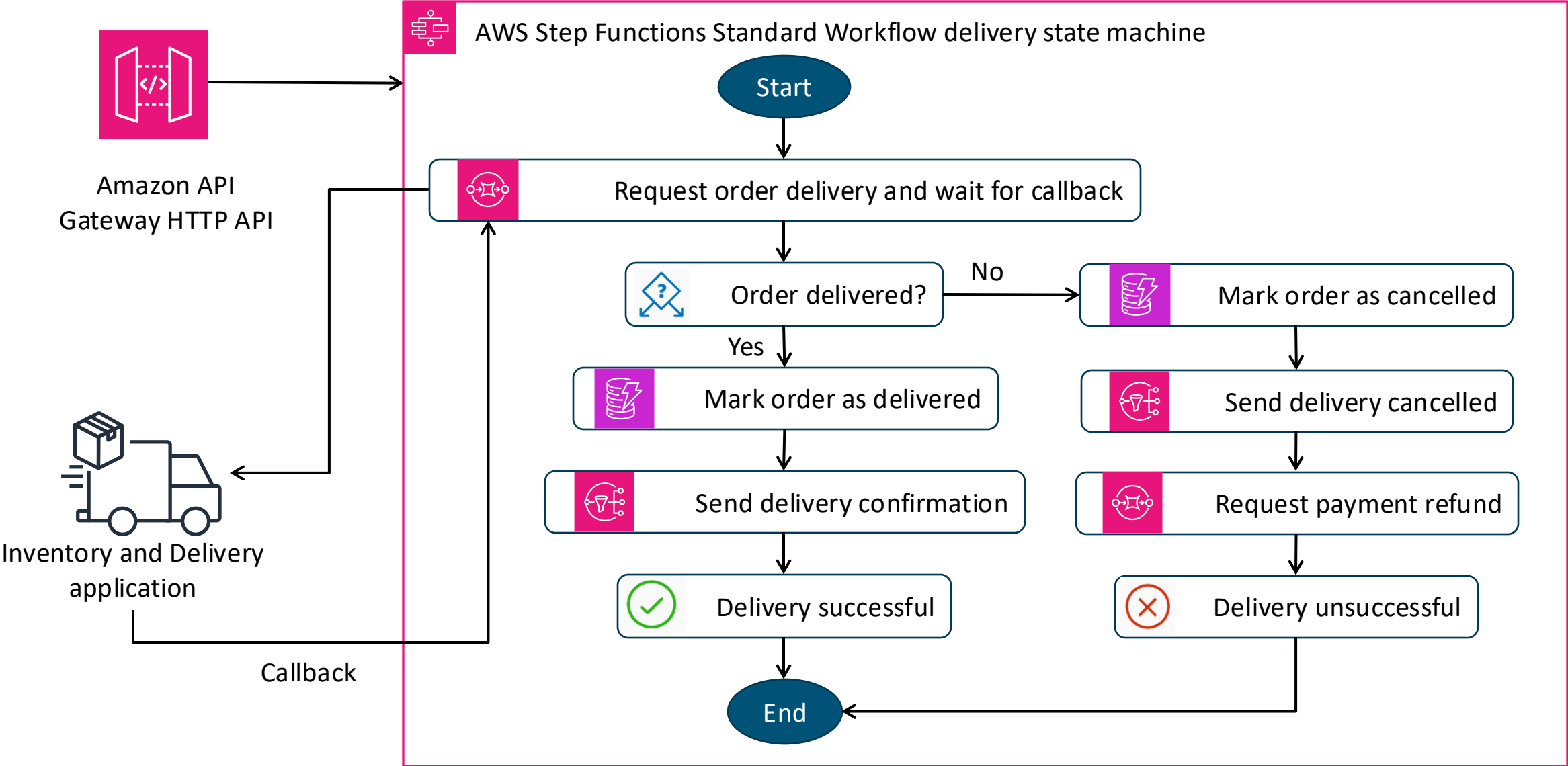
Solution: Shopping cart microservice



Solution: Payment microservice



Solution: Delivery workflow service



Key takeaways: Extending serverless architectures with Amazon API Gateway



- Amazon API Gateway provides the ability to create, publish, and maintain application APIs.
- It provides access to AWS services and publicly accessible endpoints.
- Use REST APIs when full API management and control are required.
- Use HTTP APIs when lower latency and lower cost than REST APIs are required.
- Use WebSocket APIs for real-time applications that require an active session.



Guided lab: Breaking a Monolithic Node.js Application into Microservices (Microservices lab)

Microservices lab tasks:



- In this lab, you will do the following:
 - Migrate a monolithic Node.js application to run in a Docker container.
 - Refactor a Node.js application from a monolithic design to a microservices architecture.
 - Deploy a containerized Node.js microservices application to Amazon ECS.
- Open your lab environment to start the lab and find additional details about the tasks that you will perform.

Debrief: Microservices lab

- How did you deploy the containerized monolithic application to an Amazon ECS runtime environment?
- What did you learn about task definitions in this lab?



Applying AWS Well-Architected Framework principles to microservices and serverless architectures

Building Serverless Architectures and Microservices

AWS Well-Architected Serverless Applications Lens



Reliability



Security



Performance
Efficiency



Cost
Optimization

Best practice approach: Failure management



Reliability

Best practices

Use a dead-letter queue mechanism to retain, investigate, and retry failed transactions.

Roll back failed transactions.

Best practice approach: Identity and access management



Security

Best practices

Control access to your serverless API.

Control access to your serverless application.

Best practice approach: Data protection



Security

Best practice

Encrypt data in transit and at rest.

Implement application security in your workload.

Best practice approach: Selection



Performance
Efficiency

Best practice

Optimize the performance of your serverless application.

Best practice approach: Cost-effective resources



Cost
Optimization

Best practice

Optimize application cost.

Best practice approach: Optimizing over time



Cost
Optimization

Best practice

Use direct AWS service integrations.

Key takeaways: Applying AWS Well- Architected Framework to serverless architecture



- Use a dead-letter queue mechanism to retain, investigate, and retry failed transactions.
- Roll back failed transactions.
- Control access to your serverless API.
- Control access to your serverless application.
- Encrypt data in transit and at rest.
- Implement application security in your workload.
- Optimize the performance of your serverless application.
- Optimize application cost.
- Use direct AWS service integrations where available.



Café lab: Implementing a Serverless Architecture for the Café (Serverless Café lab)

The evolving café architecture: version 7

Architecture Version	Business reason for update	Technical requirements/ architecture update
V1	Static website for small business.	Host the website on Amazon S3.
V2	Add online ordering.	Deploy web application and database on Amazon EC2.
V3	Reduce effort to maintain the database and secure its data.	Separate web and database layers. Migrate database to Amazon RDS on a private subnet.
V4	Enhance the security of the web application.	Use Amazon VPC features to configure and secure public and private subnets.
V5	Ensure the website can handle an expected increase in traffic and remain highly available and resilient to failure.	Add a load balancer, implement auto scaling on the EC2 instances and distribute compute and database instances across 2 availability zones.
V6	Consistently deploy, manage, and update café resources across Regions.	Build a version-controlled CloudFormation template to deploy the network and application layers. Deploy the CloudFormation stack to another Region.
V7	Improve performance and reduce costs while supporting reporting requirements.	Deploy Lambda functions that connect to the Amazon RDS database and generate a report based on a schedule.



Serverless Café lab tasks



- In this lab, you will do the following:
 - Implement a serverless architecture to generate a daily sales report.
- Open your lab environment to start the lab and find additional details about the tasks that you will perform.

Debrief: Serverless Café lab

- When you created the DataExtractor Lambda function, how did you configure it to extract data from the Amazon RDS database?
- What did you do to configure the salesAnalysisReport function to run daily at a specified time?



Module wrap-up

Building Serverless Architectures and Microservices

Module summary

This module prepared you to do the following:

- Define serverless architectures.
- Identify the characteristics of microservices.
- Architect a serverless solution with AWS Lambda.
- Define how containers are used in AWS.
- Describe the types of workflows that AWS Step Functions supports.
- Describe a common architecture for Amazon API Gateway.
- Use the AWS Well-Architected Framework principles when building serverless architectures.

Considerations for the café

- Discuss how the café lab in this module answered the key questions and decisions that were presented at the start of this module for the café business.



Module knowledge check



- The knowledge check is delivered online within your course.
- The knowledge check includes 10 questions based on that material that was presented on the slides and in the slide notes.
- You can retake the knowledge check as many times as you like.

Sample exam question

What is the most effective use of Amazon Elastic Container Service (Amazon ECS) when refactoring a monolithic application to use a microservice architecture?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- Amazon ECS
- refactoring a monolith
- microservice architecture

Sample exam question: Response choices

What is the most effective use of Amazon Elastic Container Service (**Amazon ECS**) when **refactoring a monolithic** application to use a **microservice architecture**?

Choice	Response
A	Create services that each provide a distinct function of the application, and run multiple services in a single container that Amazon ECS manages.
B	Port the application to a new image, and run it in a container that Amazon ECS manages.
C	Refactor the application and centralize common functions to create a smaller code footprint.
D	Create services that each provide a distinct function of the application, and run each service in a separate container that Amazon ECS manages.

Sample exam question: Answer

The answer is D.

Choice	Response
D	Create services that each provide a distinct function of the application, and run each service in a separate container that Amazon ECS manages.



Thank you

Corrections, feedback, or other questions?

Contact us at <https://support.aws.amazon.com/#/contacts/aws-academy>.